

CHALLENGE TRACK 2 · QUANTUM OPTIMIZATION / QML

Choosing Wisely: Optimization Under Constraints

From a first QAOA cut to a constrained, real-world portfolio at scale.

Background and Motivation

Combinatorial optimization — picking the best option from an exponentially large set of discrete choices — sits behind countless problems in finance, logistics, and scheduling. Because the number of candidates explodes with problem size, exact classical methods struggle, and good approximate solvers are worth a great deal. Quantum computing offers a distinctive angle through the **Quantum Approximate Optimization Algorithm (QAOA)**, introduced by Farhi, Goldstone and Gutmann in 2014: encode the objective into a cost Hamiltonian whose lowest-energy state is the best solution, then use a shallow, alternating quantum circuit and a classical optimizer to search for it.

A decade later, QAOA-style methods sit at the heart of utility-scale optimization on IBM hardware. Error-suppressed pipelines such as that of Sachdeva et al. (2024) have pushed nontrivial binary optimization to the 100+ qubit scale — the inspiration for the summit of this track. Marking ten years of cloud quantum computing, this challenge takes participants from a textbook graph problem to a constrained, real-world optimization task, and lets them choose how to attack it.

> **The scientific question**
>
> How well can a quantum algorithm solve a constrained, real-world optimization problem — and what does it take, in encoding, algorithm choice, and error handling, to keep working as the problem grows?

The problem: constrained portfolio selection

The tutorial notebook solves **Max-Cut**, the standard QAOA warm-up, on a small graph. The challenge itself uses a constrained, real-world problem where the *encoding* is the heart of the task: **portfolio selection**.

We are given n candidate assets. Each asset i has an expected return μ_i, and each pair (i, j) has a covariance σ_{ij} describing joint risk. We choose a binary vector $\mathbf{x} \in \{0,1\}^n$, where $x_i = 1$ means "include asset i". The mean–variance objective to *maximize* is

$$f(\mathbf{x}) \;=\; \sum_{i=1}^{n} \mu_i \, x_i \;-\; q \sum_{i,j} \sigma_{ij} \, x_i x_j, \tag{1}$$

where $q \geq 0$ is a risk-aversion parameter that you choose. The selection is subject to a **budget constraint** fixing the number of chosen assets to K:

$$\sum_{i=1}^{n} x_i \;=\; K. \tag{2}$$

To solve this with QAOA, the constraint (2) is folded into the objective as a quadratic **penalty**, giving an unconstrained cost function

$$C(\mathbf{x}) \;=\; -\,f(\mathbf{x}) \;+\; \lambda\Big(\sum_{i=1}^{n} x_i - K\Big)^2, \tag{3}$$

with a penalty strength $\lambda > 0$ large enough that infeasible selections are never optimal. Equation (3) is a QUBO; substituting $x_i = (1 - Z_i)/2$ turns it into an Ising cost Hamiltonian $\hat{H}_C$ whose ground state encodes the optimal portfolio — exactly the object QAOA minimizes. Choosing q, λ, and the encoding well is a central part of the challenge.

Getting Started — Beginner

Your target

Solve a portfolio-selection instance of $n = 6$–8 assets, choosing exactly K of them (Eqs. (1)–(2)), with QAOA on a noiseless simulator. Encode the budget constraint as the penalty term of Eq. (3). Deliver:

(a) the selected portfolio and its approximation ratio against the brute-force optimum;
(b) a plot of approximation ratio versus QAOA depth p;
(c) a study of how the penalty strength λ affects whether the returned solutions are feasible.

Suggested milestones

These are optional pointers, not requirements — how you reach the target is up to you.

- Build the QUBO of Eq. (3) and map it to an Ising cost Hamiltonian.
- Solve with QAOA following the tutorial workflow, and verify against brute force (feasible for $n \lesssim 20$).
- Increase the depth p and watch the approximation ratio improve.
- Sweep λ and observe the transition from infeasible to feasible optima.

Next Steps — Intermediate and Advanced

The two levels below are open-ended: the target defines *what* to achieve, and the milestones merely suggest *how*. The intermediate level additionally asks you to choose your algorithmic approach.

Intermediate — Choose Your Approach

Your target

Solve a larger, more tightly constrained instance of Eqs. (1)–(2) ($n \approx 12$–20 assets) and beat a sensible classical baseline (e.g. a greedy heuristic) on solution quality. You must commit to *one* route and justify the choice with your results:

- **Variational / QAOA route:** warm-starting, constraint-preserving or custom mixers, better optimizers, smarter initialization.
- **Quantum machine learning route:** a variational model trained to produce high-quality feasible solutions, or a QML approach to the underlying selection structure.

Suggested milestones

- If QAOA: try warm-starting from the continuous relaxation, or a mixer that preserves the budget (2).
- If QML: define your model, training data, and loss, and explain what "learning" buys you here.
- Either way: benchmark against the classical baseline and report where the quantum approach helps or hurts.

Advanced — Push Toward Utility Scale

Your target

Tackle an instance of Eq. (3) (portfolio, or a related binary optimization problem) large enough to require real hardware and to expose scaling obstacles — on the order of $n \gtrsim 25$ variables — and produce good-quality solutions despite them. Demonstrate that you have confronted trainability and/or noise (e.g. via error suppression and mitigation), and report the largest instance whose solution quality you can defend.

Suggested milestones

- Confront barren plateaus / trainability as circuits deepen or widen, and show how you mitigate them.
- Handle higher-order (HUBO) terms or multiple constraints in the encoding.
- Report the largest instance for which you can defend the solution quality, and how you achieved it.

Running on real hardware (all levels)

On a real device (such as `ibm_basquecountry`), the two-qubit cost terms of $\hat{H}_C$ must be mapped onto heavy-hex connectivity; terms between non-adjacent qubits force SWAPs that deepen the circuit, so SWAP networks and commuting-gate routing become valuable. Finite shots and noise blur the sampled distribution, making error suppression and mitigation part of getting a trustworthy answer.

Suggested Resources

QAOA and quantum optimization in Qiskit

Quantum approximate optimization algorithm (IBM tutorial)
End-to-end QAOA for Max-Cut, from a small graph to a utility-scale problem on hardware — closest to the tutorial notebook.

Advanced techniques for QAOA
SWAP networks, commuting-gate routing, and depth reduction — relevant to the advanced level.

QAOAAnsatz documentation
Reference for the QAOA circuit class, including mixer and initial-state options (useful for custom mixers).

Qiskit Optimization documentation
Building QuadraticPrograms, encoding constraints, and converting problems to Ising Hamiltonians.

Portfolio optimization tutorial
Encoding a budget-constrained portfolio-selection problem for variational solvers — directly relevant to the challenge problem.

Encoding, constraints, and optimizers

Converters for quadratic programs
How constraints become penalty terms in the cost Hamiltonian — the core encoding skill for this track.

Warm-starting QAOA
Initializing QAOA from a classical relaxation — a useful lever for the intermediate level.

Hardware, transpilation, and error mitigation

Transpile with pass managers
Transpiling circuits for a backend and its heavy-hex connectivity.

Error mitigation and suppression techniques
Overview of the resilience options available through Qiskit Runtime.

Background reading

Farhi, Goldstone & Gutmann, A Quantum Approximate Optimization Algorithm (2014)
The paper that introduced QAOA — the foundation of this track.

Sachdeva et al., Quantum optimization on a 127-qubit gate-model IBM quantum computer (2024)
An error-suppressed QAOA-style pipeline for nontrivial binary optimization at utility scale — inspiration for the advanced level.